# Game Proposal: StackOverflow

CPSC 427 – Video Game Programming

## Team: Team ΛSIMOV

Thomas Gray 59527523

Dean Yang 67057695

Sky Huang 38929873

Amanda Tian 49593320

Vincent Gilbert 67092841

### Story:

*Briefly describe the overall game structure with a possible background story or motivation. Focus on the gameplay elements of the game over the background story.*

The game begins with the player character, a robot, waking up in a corner of a lab. When the robot first wakes up, it is greeted by a call from the scientist via a dialogue box. He introduces himself as a scientist who has been trapped in a lab overrun with rogue robots. Although he's unable to escape himself, he's glad he's found a robot that hasn't gone rogue and offers to remotely guide the robot to the lab exit. However, in order to traverse through the lab, he warns that the robot must defeat all enemy robots in each room before gaining clearance to move onto the next room. He then reassures the robot that it's equipped with the tools to do this, explaining to the player how to move, dash, shoot bullets, and make use of the bullet stacking mechanic. Finally, the scientist ends the call, directing the robot to navigate through the first area of the lab. Once the robot reaches that checkpoint (i.e. after defeating the first boss), he'll contact the robot again for more directions.

As the player traverses through the lab, they are encouraged to interact with objects (such as lab equipment, lab notes, etc.) and read about them to learn more about the lab, the origins of the robot, and the true intentions of the scientist.

Once the player reaches the lab exit at the very end, they will be blocked off by the scientist. He reveals that he had been testing the robot's combat ability by pitting it against the lab's combat robots. Although he's proud that the robot made it to the end, he declares himself as the final test before the robot is deemed strong enough to survive the world outside, which is overrun by even more combat robots. The player must then fight the scientist as the final boss.

Once the fight is over and the scientist is weakened, the player can either: a) deal the last blow and kill him, then take the key from him to leave the lab or b) forgive him and convince him to leave the lab together with the robot.

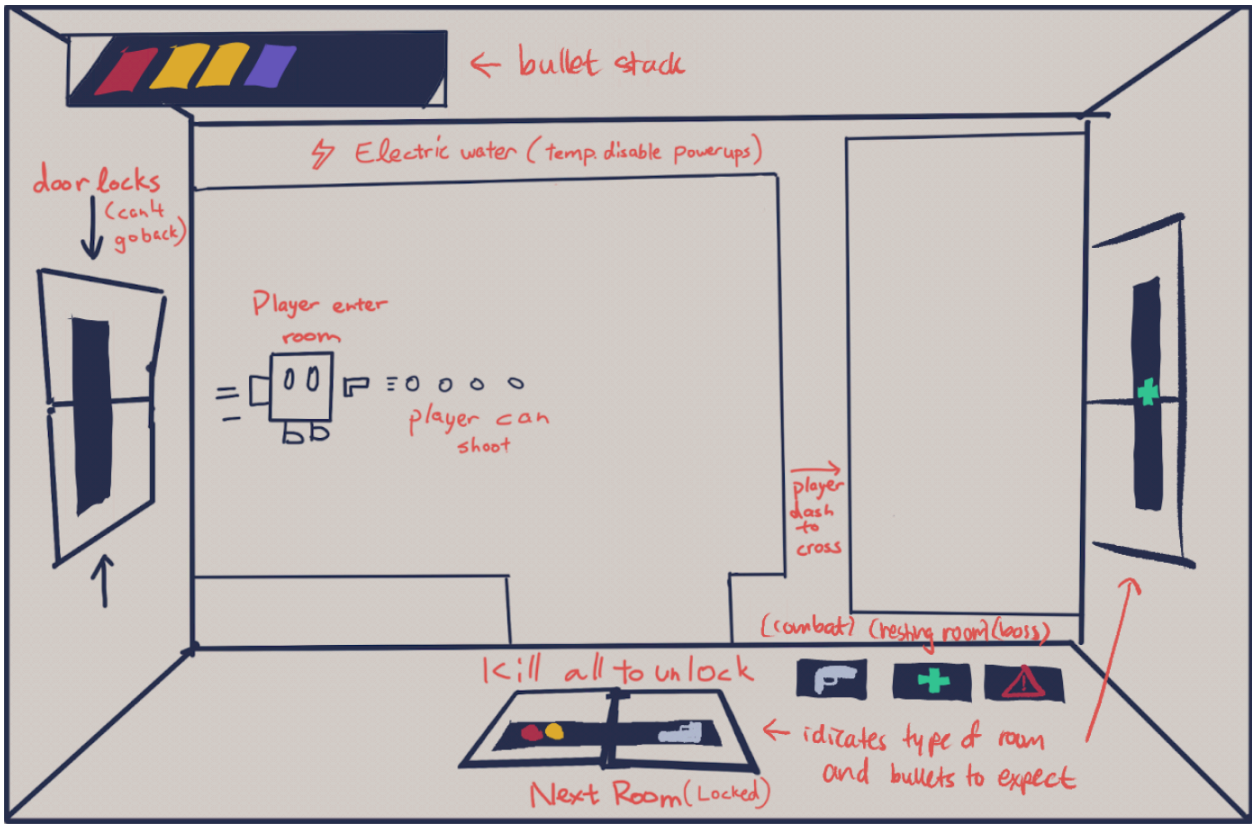
## Scenes:

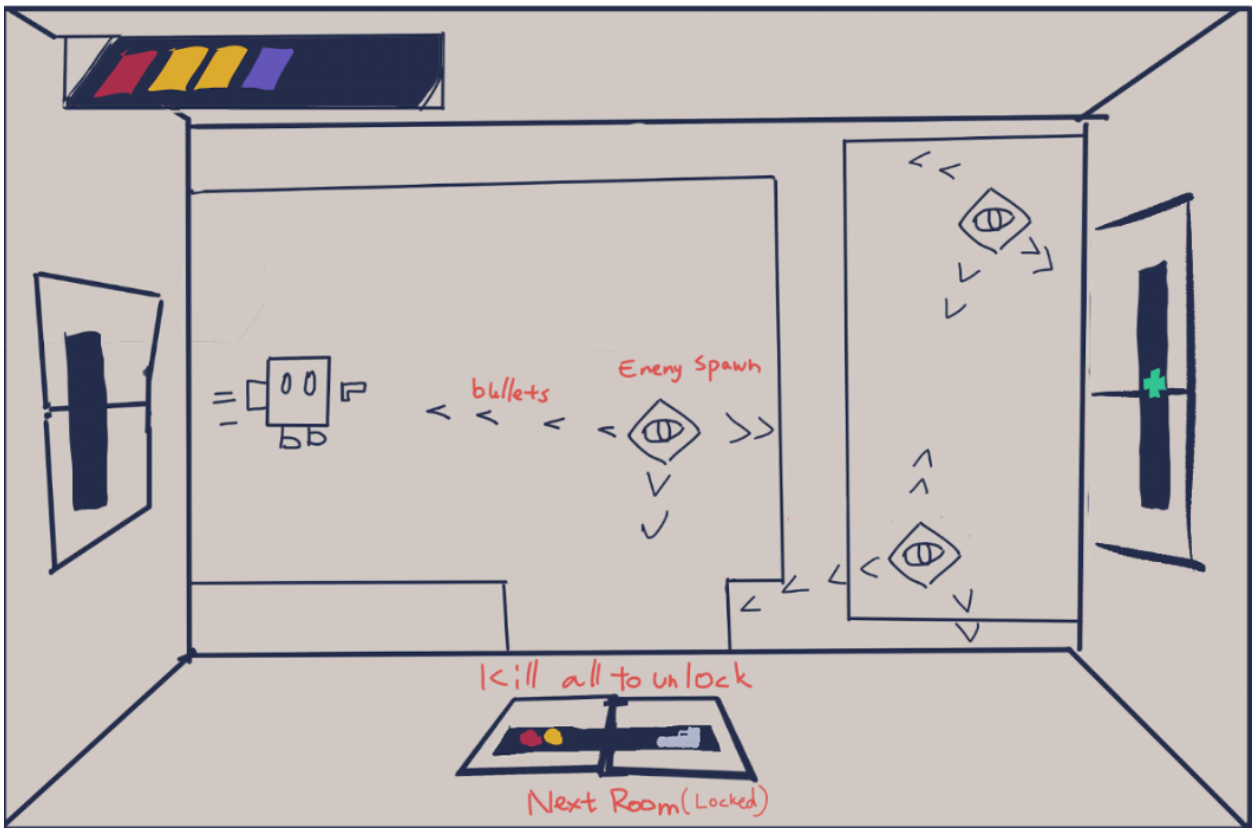
*Produce basic, yet descriptive, sketches of the major game states (screens or scenes). These should be consistent with the game design elements, and help you assess the amount of work to be done. These should clearly show how players will interact with the game and what the outcomes of their interactions will be. For example, jumping onto platforms, shooting projectiles, enemy pathfinding or 'seeing' the player. This section is meant to demonstrate how the game will play and feeds into the technical and advanced technical element sections below. If taking inspiration from other games, you can include annotated screenshots that capture the gameplay elements you are planning to copy.*

### Main gameplay:

### Standard enemy room:

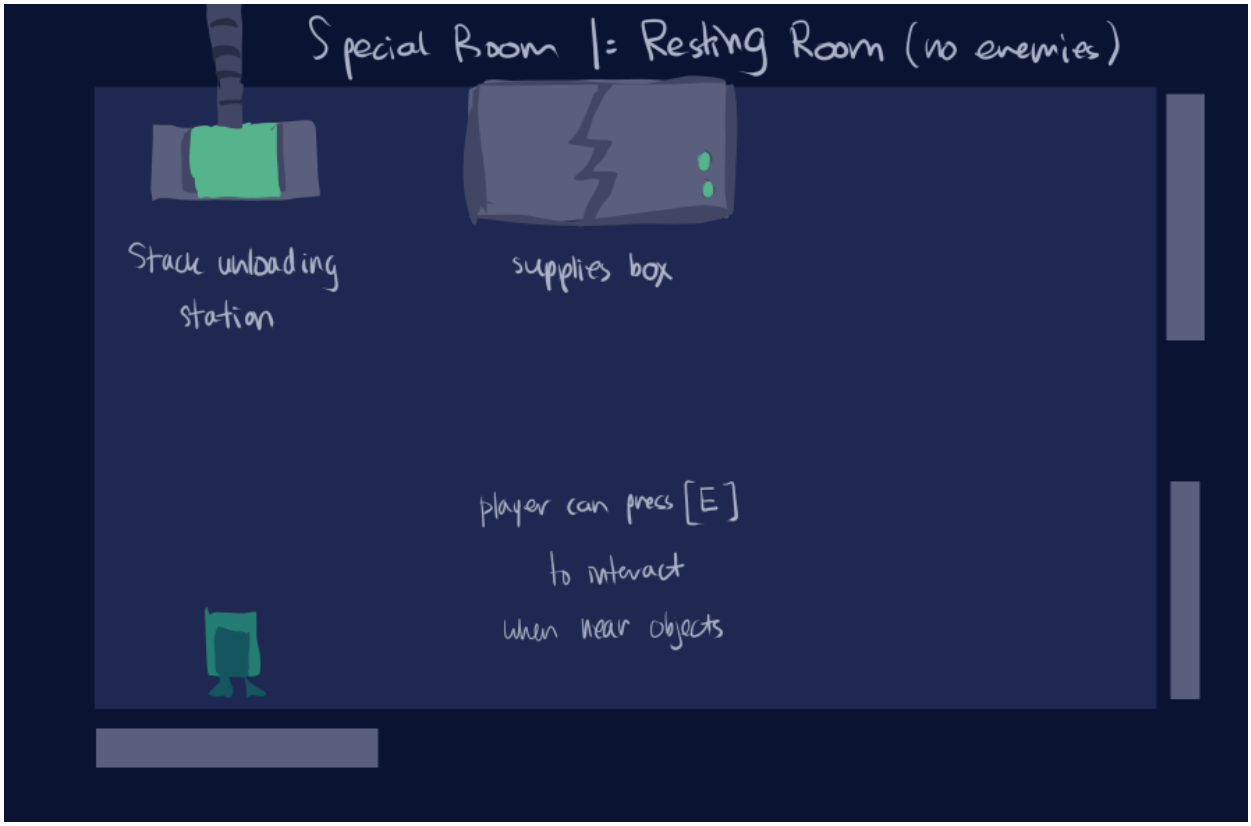
In standard enemy rooms, players must kill all enemies before moving onto the next room. All enemies spawn at once shortly after the player enters the room.



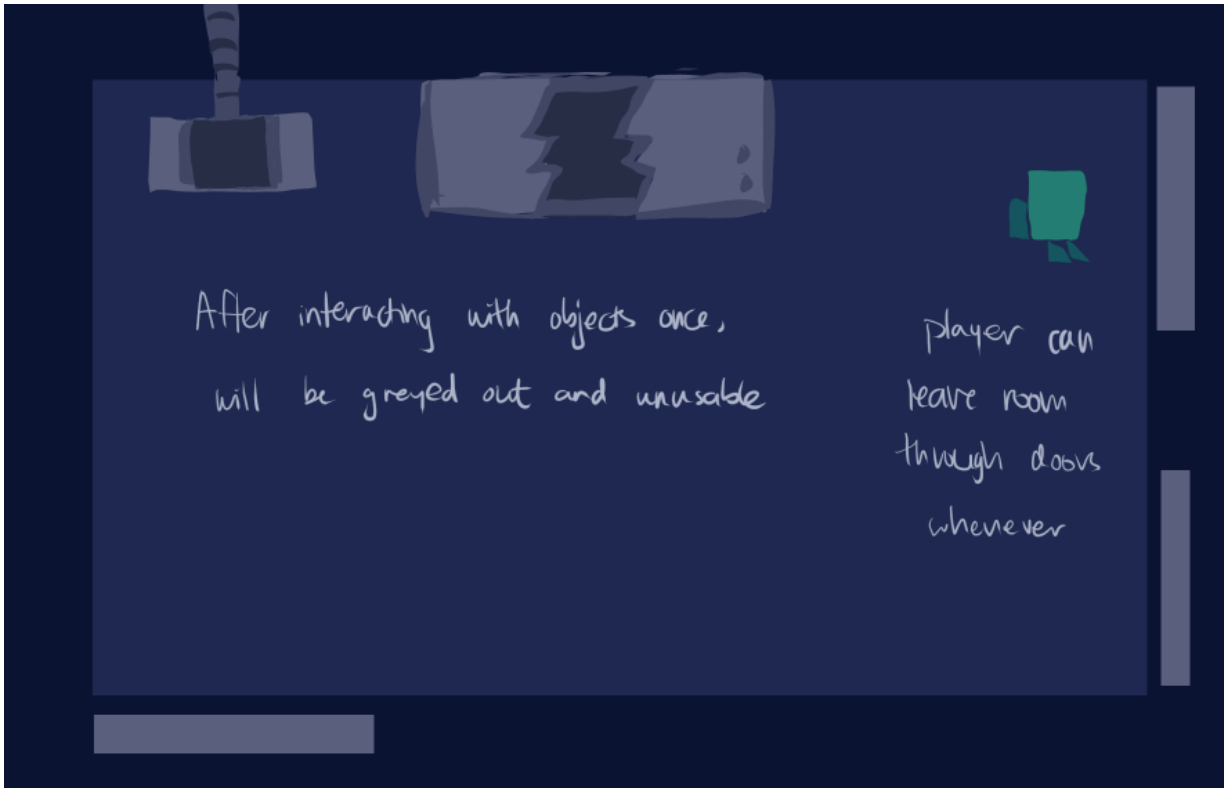



## Resting room:

Resting rooms have no enemies. They allow the player to pop the top X number of bullets off of their stack at the stack unloading station, or give the player a random buffing bullet through the supplies box.

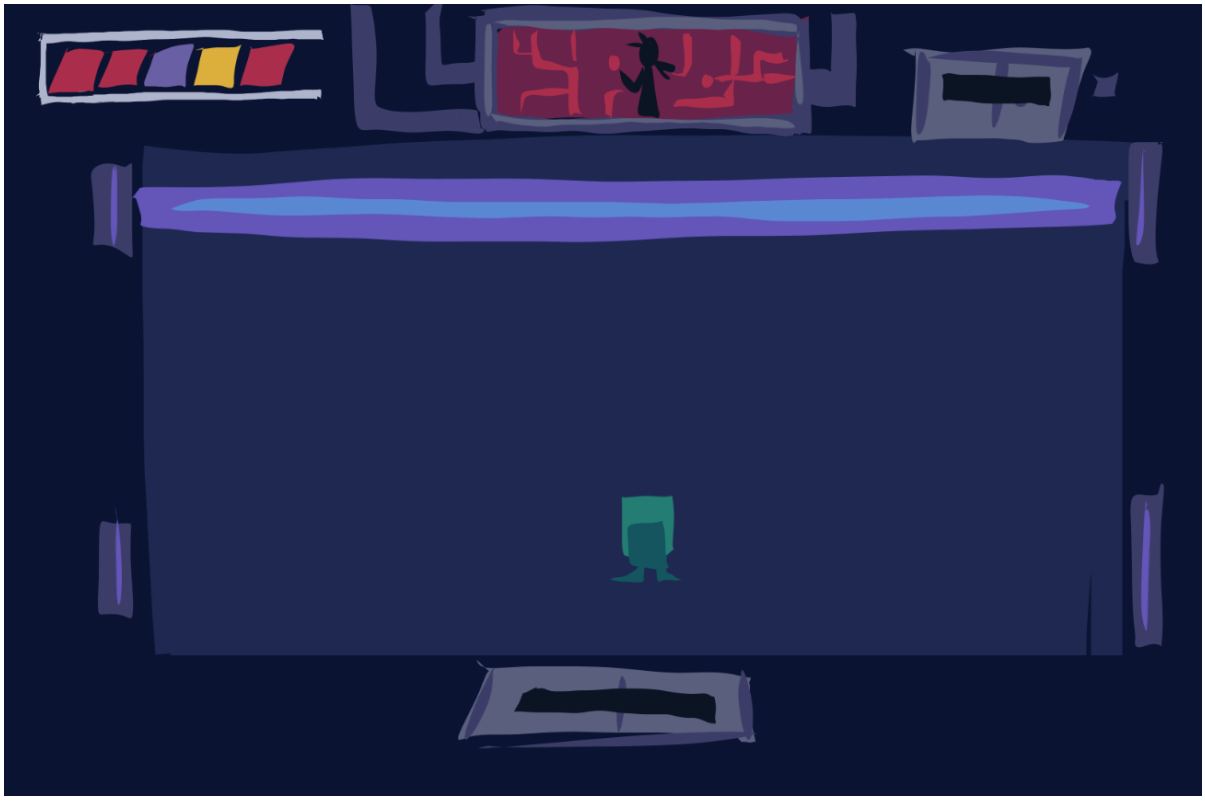

Special Room |: Resting Room (no enemies)
Stack unloading station
supplies box
player can press [E] to interact when near objects


lets players remove top X bullets off stack
Players can get X random buffing bullets

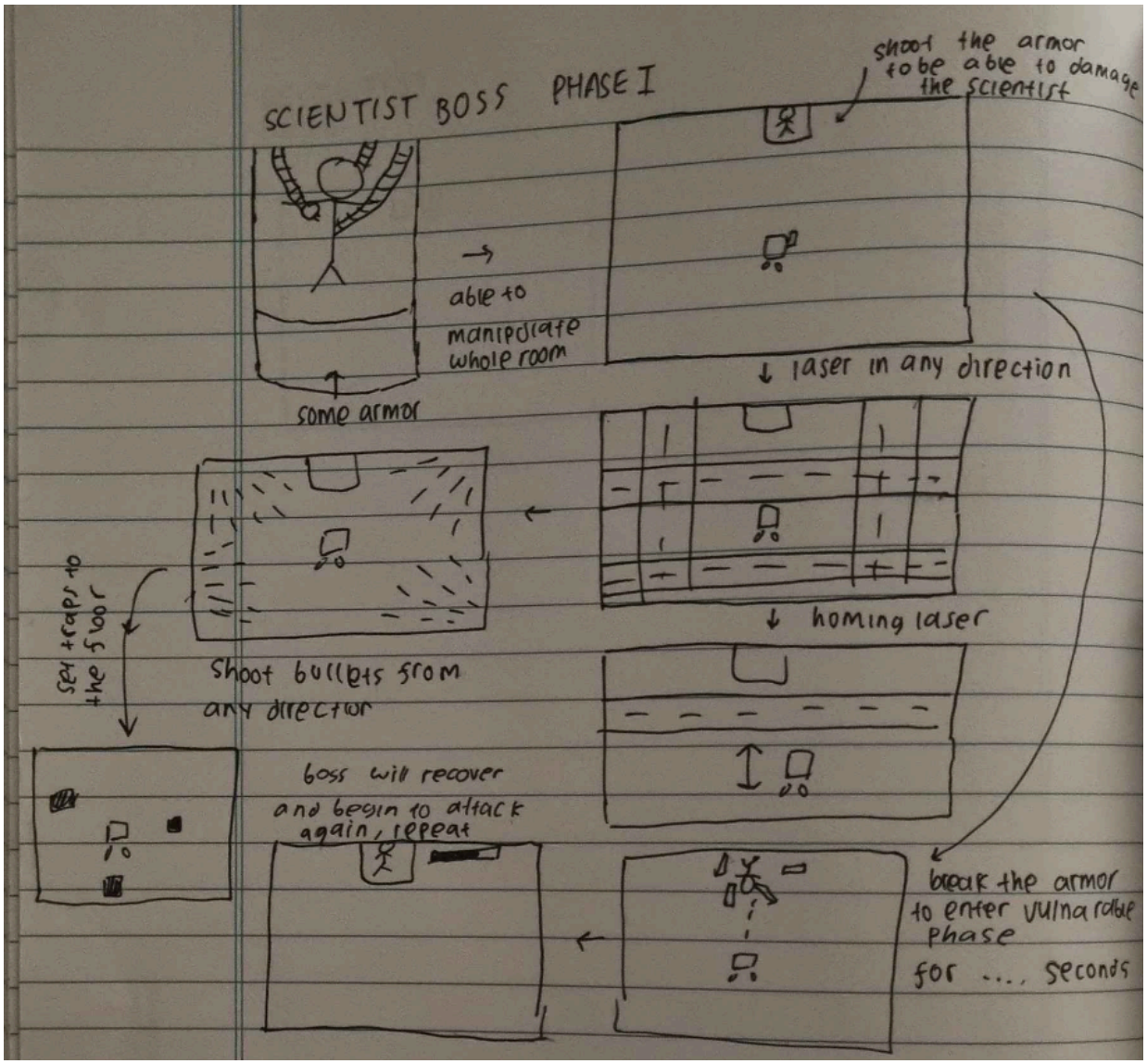



## Boss rooms

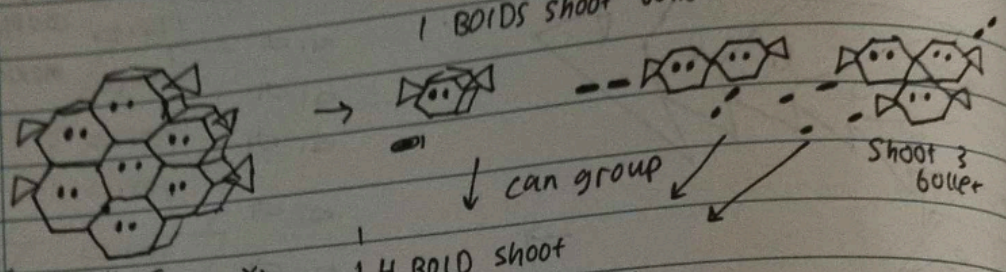
Boss rooms are similar to standard enemy rooms, but instead have a single boss the player has to defeat. May also have a specialised room design. Below is depicting a concept for the boss fight against the scientist who controls lasers in the room to fight the player.



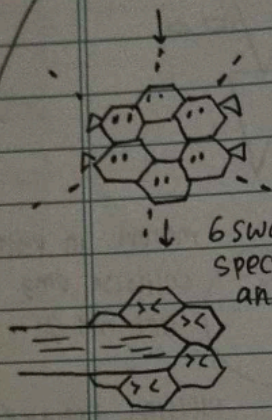
## Enemy AI


SCIENTIST BOSS PHASE I
shoot the armor to be able to damage the scientist
able to manipulate whole room
some armor
laser in any direction
homing laser
Shoot bullets from any direction
set traps to the floor
boss will recover and begin to attack again, repeat
break the armor to enter vulnarable phase for ... seconds

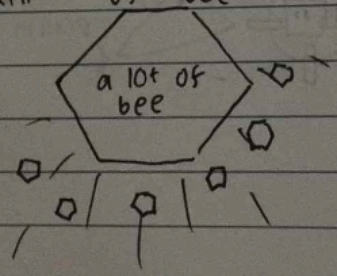
BEE HIVE BOIDS YELLOW




Ultimate skill ?

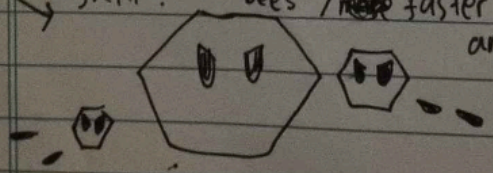



skill : can produce a lot of bee

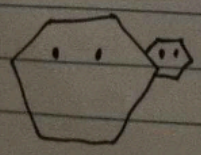



→ mini be can combine each othe ~~immediatlely~~

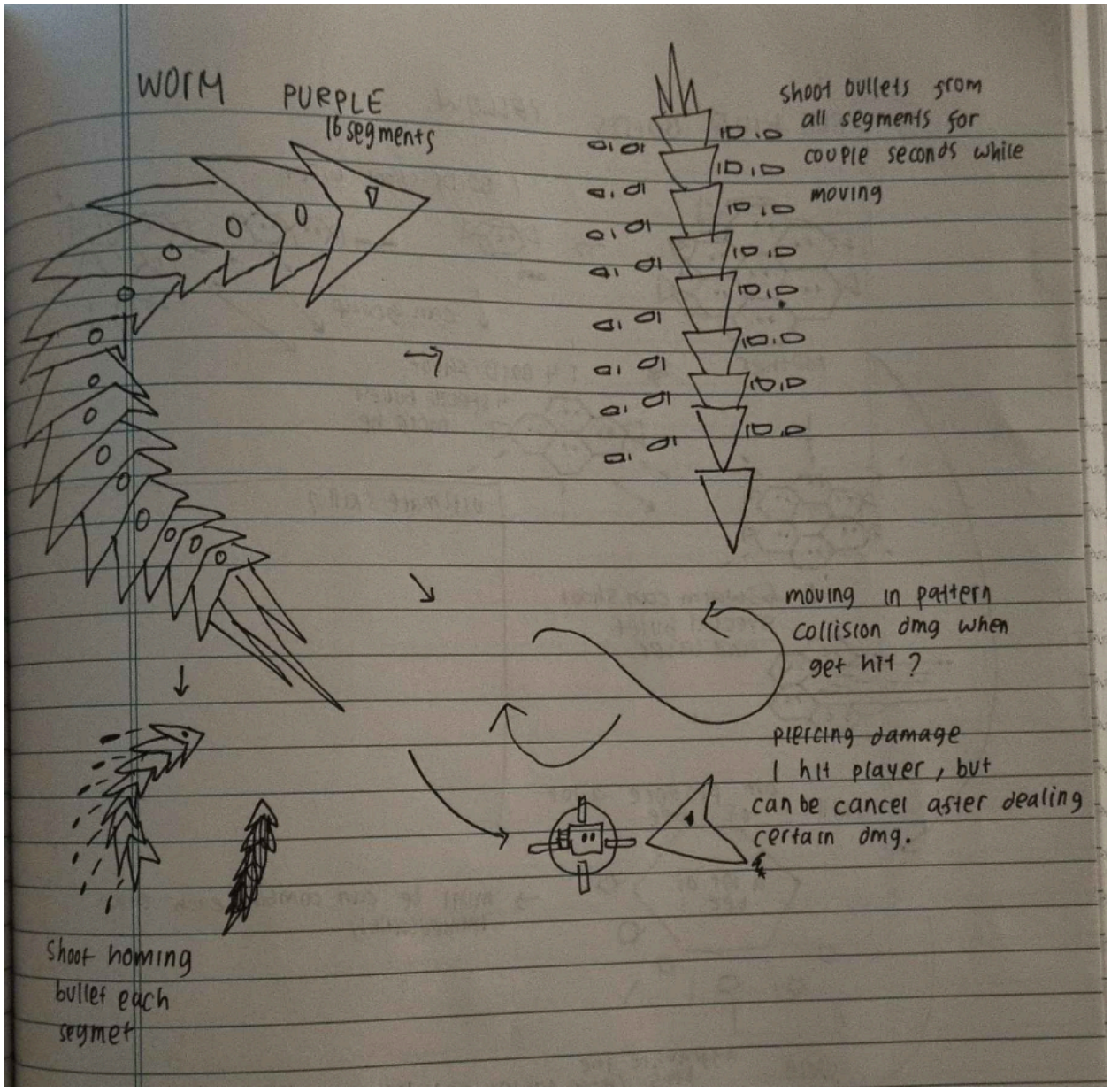
skill : aggravate the bees / ~~more~~ faster and deal more dmg for certain amount of time



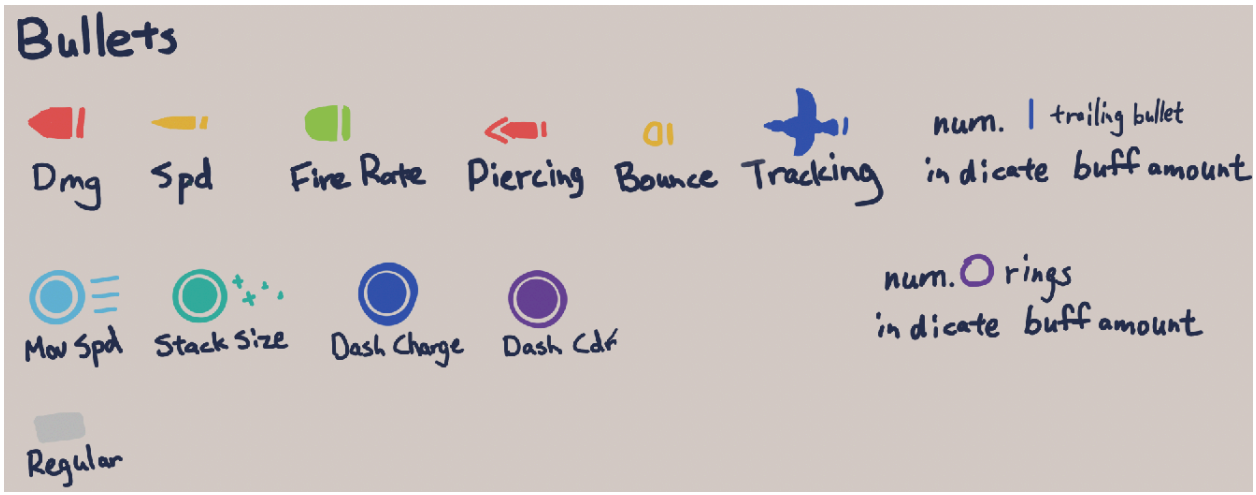
skill : sels heal



mini bee can regroup and heal the mother


WORM
PURPLE
16 segments
shoot bullets from all segments for couple seconds while moving
moving in pattern
collision dmg when get hit ?
piercing damage
1 hit player, but can be cancel after dealing certain dmg.
shoot homing bullet each segmet

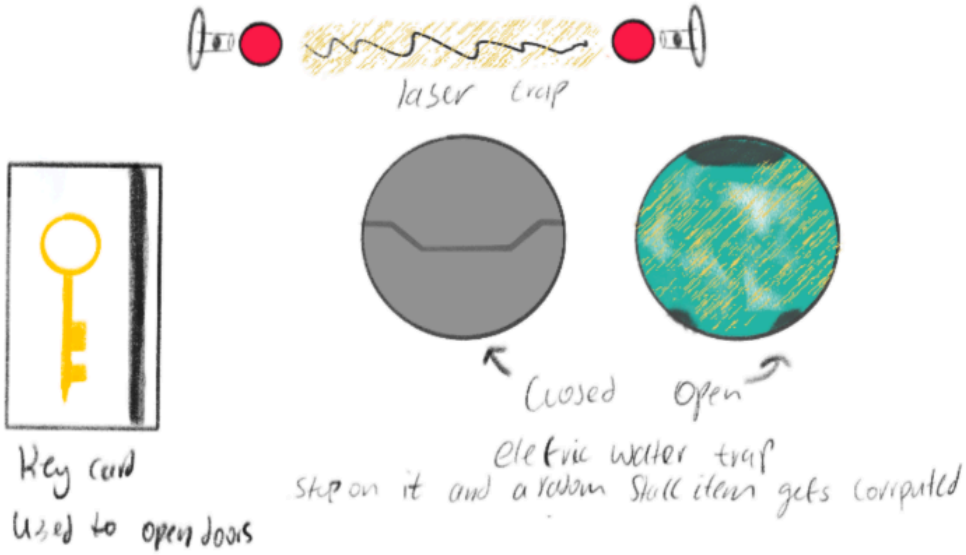
## Bullets




## Dialogue


## Traps and Keycard

Additional features we may implement if time allows it. The keycard is an item that can be picked up in the lab to access certain locked rooms, and traps can “damage” the player.




## Technical Elements:

*Identify how the game satisfies the core technical requirements: rendering; geometric/sprite/other assets; 2D geometry manipulation (transformation, collisions, etc.); gameplay logic/AI, physics.*

The full game will feature 3 main regions with 1 boss for each region. Each region will also have its own enemies and music. For the scope of this course, we plan to start with 2 regions, and add more if time permits it (or remove the second region if time does not allow).

### Assets (geometry, sprites, audio, etc.)

We will make most of the visual assets ourselves, including the title screen, sprites of the player, enemies, NPCs, UI, rooms and other objects that exist in the rooms. If we're short on time, we will use assets from online (such as from kenny.nl), notably for visuals of the rooms. Since we are designing assets ourselves, we have agreed upon the following design constraints to encourage everyone on the team to contribute, while still maintaining a unified game look:

- All assets will be designed with the following colours:

- All assets will be in the Text Mode style (with lvllvl.com being our Text Mode editor of choice). In this editor, we will be using modular shapes in vector mode.
- All assets should be proportionate to the player sprite, which is 8 tiles wide.

- Background assets (floor, walls, background objects) should be darker coloured while all other entities should be lighter-coloured to make them easily visible.

We plan to get most of our audio assets from an online open-source in addition to designing some of our own sounds.

## Rendering

Shaders will be used for visual effects, such as CRT scan lines, bloom, chromatic aberration, and glow effects. These will give our game the intended tech-y appearance.

We have considered two ways of implementing sprites:

- Method 1:
  - import sprite (as vector image) into Blender to get a shape that covers sprite
  - export vertex map from Blender into OpenGL
  - load in vector image as texture
- Method 2:
  - every sprite is a box that just has texture that shows shape, indicating transparency

Since the game is not side scrolling and we do not plan for the camera to move, parallaxing effects will not be necessary.

## 2D geometry manipulation (transformation, collisions, etc.)

There will be collision detection between the following entities:

- Player and enemy: when the player touches an enemy, they will gain a “blank” bullet added to the stack (and takes up 1 stack space), with no stat modifiers attached to it (contact damage)
- Player and bullet: when the player touches an enemy bullet, they will gain a bullet added to the stack with corresponding effects based on the bullet type shot. Typically, bullets take up 1 space on the stack, but bullets with stronger buffs may take up multiple slots. Enemy bullets usually disappear after coming in contact with the player, but some bullet types (eg: lasers) will persist longer.
- Player/bullets/enemies and walls of room: to keep the player, enemies and bullets contained in the room. The player and enemies will be blocked by walls, while bullets will either disappear (typical bullets), bounce (bouncing bullets), or stop (lasers).
- Player and interactable objects: when the player approaches certain objects, they can press E to interact with them.

Note that we will not have collision detection between enemies.

We will use the following collision detection methods:

- Circle collision (radius + centre)
- Line segment collision (start + end)
- Axis-aligned bounding box (AABB) collision (top right corner + bottom left corner for room)

Moving the player, enemies and bullets across the screen will require sprite translations.

## Gameplay logic (world interaction, character interaction, player controls, etc.)

**Controls**

The player controls the movement and combat of the main character by using WASD to move and pressing and holding the LMB to shoot in the direction of their cursor. Additionally, the player robot can dash. Dashing allows the player to quickly move 3 units (floor tiles) in the direction designated by WASD. Dashing has a cooldown, so the player must wait a few seconds before being able to dash again. Initially, the player starts with one dash charge, but can get more through certain bullet buffs.

**Interactable Objects and NPCs**

Additionally, the player can interact with objects and NPCs throughout the lab by pressing E. Doing so will open up text in the dialogue box, with the aim of further describing the world the robot is in. While a dialogue box is open, pressing E will also proceed to the next dialogue. When options are presented in the dialogue, the player can click on different options to choose them. Note that interactable objects in standard enemy rooms cannot be interacted with until the player clears the room, should there be any.

The dialogue system is also used for other NPC interactions, and is the main way for the scientist to communicate with the player.

**Rooms**

Rooms are premade and are procedurally linked, with all rooms in a shared region following a different visual theme. To simplify room collision detection, all rooms will be of the same size and shape. However, they may differ in appearance, enemy types, enemy spawn formations, and objects placed in the room. Each room also has doors to other rooms, each with a screen on it that indicates what the room will contain to help the player choose the next room. For now, we plan for most rooms to have two doors, with the exception of boss rooms (both the room leading to the boss room and the boss room itself can only have one door). Note that players cannot return to previous rooms.

We plan to have the following room types:

- Standard enemy room: a room with normal enemies the player must defeat before progressing onto the next room.
- Boss room: a room with a boss the player must defeat before progressing onto the next room.
- Resting room: a room with no enemies. The player can interact with the stack unloading pad to remove the top X bullets off their stack. The room may also contain additional interactables.

**Bullets**

There exist many different types of bullets that the player can collect and put onto the bullet stack. When on the stack, each bullet may give a buff/debuff player, with bullet shape and colour indicating the type of effect:

- Shape conveys category
    - Round shapes affect the player's stats. Eg: move speed, stack size, dash charge, dash cdr
    - Sharp shapes affect player bullets. Eg: bullet speed, fire rate, dmg, bounce, pierce, tracking
- Colours conveys the type within each category
    - Red: damage
    - Yellow: speed
    - Green: fire rate
    - Blue: move speed

Outside of stat modifications, special bullets can also grant the player bullet abilities. These include:

- Bouncing bullets: these bullets will bounce off of walls but do not bounce when they hit enemies or a player. The number of times a bullet can bounce before it expires upon impact can vary (somewhere between 1 to 3). Multiple counts of this bullet will yield more bounces.
- Piercing bullets: these bullets will not expire on their first contact with an entity, and instead will continue to fly forwards. Multiple counts of this bullet will give this bullet more chances before it expires on contact.
- Tracking bullets: these bullets will home onto the player/enemy. After the bullet is shot, it will seek out the nearest entity in its tracking radius (a certain radius around the bullet) and start tracking that entity. Tracking bullets will fly forwards until an entity is found in its radius. Multiple counts of this bullet will make tracking "better" (increase tracking radius, or increase bullet turn speed).

## AI (pathfinding, entity behaviour, etc.)

As we do not have obstacles in our rooms, and our rooms are all easy to navigate through (rectangular), we do not require pathfinding for our enemies. Our enemies will instead have more challenging attack patterns that could be otherwise difficult to achieve in variably sized rooms or rooms with obstacles, such as swarming enemies (BOIDs) and worm-like enemies. How enemies will shoot bullets will be implemented by a bullet spawning system that can spawn bullets in different patterns.

**Low level enemies:**

- Stationary enemy that shoots every 0.5 seconds towards the player.
- Fast moving enemy that moves in random directions. Shoots 5 bullets in all directions when it hits a wall (low range)
- An enemy that slowly moves towards the player and has a shotgun-like attack. Shoots infrequently.

**Medium level enemies:**

- Stationary enemy that shoots 4 bullets that home on where the player was when shot (arcing shot)
- Stationary enemy that shoots 2 slow moving homing bullets
- Enemy that moves around, shoots 2 arcing bullets and 1 big bullet towards player
- Enemy that splits into 2 smaller enemies on death
- Enemy that heals other enemies
- Enemy that shoots electric sparks. When the player walks into these puddles, one bullet on the stack will be randomly changed to another bullet type. (Alternative: removes a random bullet from the player's stack)

**High level enemies:**

- Randomly moving enemy that shoots 3 rounds of all direction bullets, with angle offsets
- Spinning laser enemy in the middle of the room
- Enemy that upon being hit can teleport toward random place

**Bosses:**

Our game will have one boss (the scientist). Should time allow, we will add up to three additional regional bosses the player must fight at the end of each region before progressing onto the next. Stack extension for finishing bosses rather than sparing them

Possible boss mechanics include:

- Boss that uses enemy stats in phase 1, player stats in phase 2 (may reserve mechanic for final boss)
- duo boss: if kill first one, second will get angrier
    - consider a laser beam linking them as an attack
    - consider resurrection mechanic: after one of the duo is down, the player has a certain amount of time to kill the other before the downed one resurrects with 25% hp

## Physics (entity interactions, particle effects, kinematics, etc.)

Bullets physics of special bullets:

- Bouncing bullets: need to calculate trajectory of bounce when it hits a wall.
- Tracking bullets: need to adjust acceleration or turn speed of bullet as it is tracking.

We will also have particle effects for bullet collisions.

## Sound (effects, music, etc.)

The general direction for our sound effects is robot-y tech sounds. Sound effects will be played when:

- When special bullets are shot
- When player / enemies get hit
- When player walks / dashes
- To signal when an enemy is about to attack

Each region and boss will have a unique music theme. For one region with two bosses and the title screen, this means we will have four different tracks. Should time permit and we make two regions and two bosses, this will be five tracks.

## Advanced Technical Elements:

*List the more advanced and additional technical elements you intend to include in the game prioritised on likelihood of inclusion. Describe the impact on the gameplay in the event of skipping each of the features and propose an alternative.*

### Advanced Enemy AI (High priority)

BOIDs as an enemy AI. Inclusion would have a visual impact, and a unique gameplay experience. For example, we could have a small BOID swarm of enemies that chooses a new entity to fly towards (enemies/player) every 3 seconds. This would also allow a hivemind/swarm boss that can:

- summons mini enemies, where the mini enemies can combine each other to make it more powerful
- mini enemies can combine back to the main BOIDS and heal the boss
- main BOIDs can shoot in all direction and spinning for 8 seconds

Skipping this feature would be acceptable if the replacement AI was interesting enough, though a BOID system would be adaptable to many different enemies by tweaking parameters. Priority: high.

Worm-like enemies/bullets, entities that follow behind a leader enemy. For example, we could have a worm enemy that shoots broadside and moves around the room while sticking close to the wall. Similar consideration to BOIDs. Priority: high.

### “Good” and “Evil” gameplay (Medium priority)

The game will adjust depending on how the player plays: “good” or “evil.” To be considered good, the player must not kill any bosses (excluding the final boss). This means that when a boss fight ends and the boss is weakened, the player must choose to talk with it through dialogue rather than deal the final blow. Although the player will lose out on possible upgrades they could have gotten from salvaging the boss’ corpse, the game will not increase in difficulty. Throughout the dialogue, the boss robot and player robot will come to a mutual understanding, and the boss will allow the player to pass through the next room.

The player will be considered evil once they kill a certain number of bosses, with every boss they kill increasing their evil counter by one. Killing bosses allows the player to scavenge their corpses and use their parts to upgrade the player, including special bullets with strong buffs or a stack size increase. As a downside, each count of evil increases the game’s difficulty by increasing enemies’ stats by one stage, so that they have higher health and shoot more punishing bullets.

The game ending may also change based on whether the player is good or evil. If the player is good, then the player robot has proven to the scientist that not all robots are mindless killing machines, and some can be reasoned with–even if they were designed to be combat robots. By extension, the scientist who is part robot is not inherently evil and deserves to live. Reinvigorated with a newfound way of solving the rogue robot crises outside in the world, the scientist and robot then head outside the lab together. Conversely, if the player is evil, then the player's only option is to kill the scientist, as there is no way to convince the scientist to allow the robot to leave.

Priority: medium. This feature would be a nice way of making the player more involved with the story, as the outcome of the story is now based on several choices they had to make, rather than a single choice they make at the end. This could also lead to a more natural resolution of the end. However, this also requires adding dialogue to bosses and a system that keeps track of counts of evil and adjusts the difficulty of the game. This may also require a "neutral" ending, where the player has killed some bosses, in order to make more sense, reserving the evil ending for when the player has killed all bosses.

Alternative: The game ending can still differ, but it instead relies on whether the player has a certain item in their stack. This item is an old journal containing the initial draft for the player robot that the scientist made and can be found and picked up in the lab. Presenting the journal to the scientist will remind him of the original intent for making the player robot: a companion bot that befriends humans. As a result, he realises that expecting the player robot alone to combat the rogue robots outside the lab would be too harsh, and he should instead work together with the robot to solve the rogue root crisis. Without the item, the player can only kill him to get across. This is easier to implement, as the journal can simply exist as a type of bullet on the stack, but it could be less satisfying.

## More Room Variants (Medium Priority)

More room variants will reward the player for more challenging play, encouraging different kinds of play, or allowing the player to explore more of the lab. Although not necessarily complicated to make, these rooms will require additional time and assets and are not a part of our core gameplay. As a result, their priority is medium.

- Special event rooms: the player will encounter a random special event when entering these rooms. No enemies will be present in these rooms, so the player must clear the special event in order to progress to the next room. So far, we have two special events in mind:
    - A room full of traps (lasers, electric water, automated turrets) that the player must dodge and manoeuvre around in order to reach the exit. At the end of the room, the player will receive a bullet with strong buffs.
      Traps include:
        - Lasers that periodically turn on and off. When hit by a laser, the player will get a "blank" bullet that gives no effects in the stack.
        - Guns in the wall that periodically shoot bullets. When hit, the player will get a bullet with corresponding buffs.

        - Electrified water puddles. Much like electric sparks, when the player walks into these puddles, one bullet on the stack will be randomly changed to another bullet type. (Alternative: removes a random bullet from the player's stack)
    - A room with an old robot NPC who the player can interact with via dialogue. He proclaims to have a special bullet and offers it to the player, in which the player can either choose to accept or decline. In actuality, this bullet has a chance to be either a "very good bullet" (a bullet with strong buffs) or a "very bad bullet" (a bullet with strong debuffs). Players must finish dialogue with the NPC robot in order to move onto the next room.
- Locked rooms: the doors to these rooms will require a keycard in order to enter. Keycards are special items that can be found across the lab, and are also shot by certain enemies as a projectile. When the player picks up a keycard (by pressing E) or gets hit by one, it will take up 1 slot on the stack. Locked rooms will have special bullets with very strong buffs.
- Elite enemy rooms: these rooms contain regular enemies in addition to elite enemies, which are significantly stronger but not as strong as bosses. Likewise, their attack patterns will also be more complex. They will give bullets with stronger buffs, with the intent of rewarding players for a challenge. These elite enemies will utilise advanced enemy AI (as described above).

### ~~Obstacles (Medium priority)~~

~~Adding static, indestructible obstacles to our rooms could better portray a cluttered lab environment with scattered equipment and boxes. Similarly, it could allow for more interesting gameplay, as players and enemies can choose to hide behind obstacles to dodge bullets. Of course, adding obstacles would require pathfinding for enemies, such as implementing the A* algorithm. More complex enemy combat AI may also not be feasible with obstacles, such as BOIDs or worm-like enemies. Additionally, collision detection will need to be done between obstacles and the player, enemies and bullets, which would behave similarly to collision detection between walls and entities. Finally, creating obstacles also means creating the assets those obstacles will use.~~

We have decided to remove obstacles completely, as they would interfere with the enemy system supporting varied enemy attack patterns.

### Dynamic Assets (Low priority)

Dynamic assets include stack visually indicated on player sprite. Low priority, purely eye-candy as the stack would be hard to read compared to the UI stack. Also may not be feasible depending on the final stack size.

## Devices:

*Explain which input devices you plan on supporting and how they map to in-game controls.*

The input devices we plan on supporting are the keyboard and mouse.

| Control | Input Device |
|---|---|
| Player movement | W / A / S / D |
| Player aiming | Mouse movement / cursor position |
| Player shooting | LMB |
| Player dash | SPACE or RMB |
| Player world interaction / Next in dialogue | E |
| Pause game / Open menu | ESC |
| Menu interaction | Mouse and LMB |

## Tools:

*Specify and motivate the libraries and tools that you plan on using except for C/C++ and OpenGL.*

~~As of now, we do not plan to use any additional libraries outside of GLFW and SDL with OpenGL. We believe these would be sufficient for our game.~~

We are now using the FreeType library to support text rendering, and ImGUI for creating a debug window that shows relevant stack information for debugging.

## Team management:

*Identify how you will assign and track tasks and describe the internal deadlines and policies you will use to meet the goals of each milestone.*

We will be using Trello to organise upcoming tasks and deadlines, as well as assign tasks to each group member. This will be based on member preferences, experience, and the number of estimated hours for the task. We will use a weekly sprint approach, ending on Monday of each week. On the same day, unfinished tasks are rolled over and new tasks are assigned.

So far, our roles are:

**Amanda:** Art lead, narrative lead, graphics

**Dean:** Main system setup, map design/progression

**Sky:** Player controls and shooting

**Vincent:** Enemy AI, projectile physics

**Tom:** Collisions, bullet stat modifying system

However, members of the team are welcome to take on tasks from other roles if need be, and roles themselves are flexible should each member's preferences change over the course of development, so long as it is discussed with the team.

## Development Plan:

*Provide a list of tasks that your team will work on for each of the weekly deadlines. Account for some testing time and potential delays, as well as describing alternative options (plan B). Include all the major features you plan on implementing (no code).*

### Milestone 1: Skeletal Game

Week 1 - player controls

- Dean
    - ECS system, main game loop
    - ImGUI debug window displaying stack information (for debugging)
    - Resized window to fullscreen
- Sky
    - player movement controls (using WASD), dashes (space)
    - Invincibility timer set for player
- Amanda
    - simple graphical representations of player, room, enemy (greyboxing)
    - UI showing player stack
    - Player sprite assets (normal, upon being hit)
    - Collision outlines drawn for circles colliders (for debugging)
- Tom
    - collision detection with player & enemy bullet, player & wall, player & enemy, player bullet & enemy
    - Collision resolution for player & wall, player & enemy bullet, player & enemy
- Vincent
    - Basic enemy framework
    - Enemy shooting

Week 2 - bullets & collision

- Tom
    - bullet stacking & stat modification system, supporting at least 2 different types of bullets and ability to remove bullets if need be
    - Bullet effects and premade bullet types
    - Enemy sprite assets
    - Bullet sprite assets
    - Basic floor asset
    - Aim indicator asset
- Sky
    - player shooting, aimed with cursor

    - Invisibility timer for bullets so that they look like they spawn outside of the player/enemy sprite (and not inside)
    - Particle effects for player bullet and wall collision, particle effects for enemies dying
- Amanda
    - ~~Basic dialogue system for scientist to communicate with player robot~~
    - ~~Game over condition and screen~~
    - Text rendering system setup
    - dialogue box with a set sequence of dialogue
    - Rough game over and game paused screen
- Vincent
    - bullet dealing damage to enemies
    - Enemy movement (random; follow player) that LERPs enemy position and rotation
- Dean
    - Addition of player hit sound
    - Game paused state and game over condition
    - Particle system setup, particle effects for dashing
    - Aim indicator that follows player's cursor movement

## Milestone 2: Minimal Playability

** Ask about what type of mesh based collision detection we can do

Week 1

- Everyone
    - sprites for one enemy, two types of bullets, room
- Tom
    - Mesh-based Collision Detection
    - bullet spawning systems that support enemies to shoot bullets in different ways (different number of bullets at once, at different intervals, in different directions, etc.)
    - Dynamically load all textures placed in the directory on during render_init (would be nice)
- Amanda
    - Fix text clipping
    - ~~Flesh out dialogue system so that each room/part of the game knows what dialogue it should display~~
    - ~~skippable, non-intrusive tutorial at start of game (plan B: text displayed in first room that tells player controls)~~
    - player sprite and animation (walking, ~~shooting~~)
    - ~~Dynamically load all textures placed in the directory on during render_init (would be nice)~~
- Dean

    - ability to move onto the next room via procedurally linking or predetermined map layout, requires to make at least one other room first
    - FPS counter, make game FPS independent
    - Instanced rendering for particle system
    - Wall and door sprites, using a special perspective shader
- Vincent
    - 3 simple enemy AI

Week 2

- Dean
    - Continue task from week 1
- Sky
    - ~~ability for player to interact with objects in the room - for clearing stacks, story panels, chests etc.~~
- Amanda
    - Flesh out dialogue system so that each room/part of the game knows what dialogue it should display
    - skippable, non-intrusive tutorial at start of game (plan B: text displayed in first room that tells player controls)
    - ~~ability to pause game~~
    - ~~display description of bullets by hovering over them while game is paused~~
    - ~~game title screen, game start function~~
    - ~~If time allows, create a script file loading system so that script writing is decoupled from programming~~
    - ~~If time allows, create a bullet pool that pre-allocates a bunch of bullets to avoid constantly creating and deleting a large number of bullets and improve performance~~
    - ~~If time allows, consider instanced rendering for bullets/enemies?~~
    - ~~If time allows, consider assigning sprites component to enemy/entity types rather than to each entity~~
- Vincent
    - add 2 more enemy variants (requires enemy sprites and AI)
    - Enemy spawning formations when entering room system
- Tom
    - bouncing/piercing/homing bullets (optional in case of delays)
    - Sound system framework; sound on event, music

## Milestone 3: Playability

Week 1

- Dean
    - add 2 more types of rooms (resting rooms, boss rooms)
- Vincent
    - scientist boss AI
- Amanda

    - display description of bullets by hovering over them while game is paused
    - sprite and animation for scientist boss
    - create objects the player can read about by interacting
    - If time allows, create a script file loading system so that script writing is decoupled from programming
- Tom
    - add a new bullet type: lightning bullet, that can corrupt stack RNG system (optional in case of delays)
- Sky
    - Additional player related mechanics (debuff states like stunned, slowed, or distracted)
    - Create 1-2 enemies

Week 2

- Dean
    - Design ~5 variants of normal enemy rooms.
- Vincent
    - continue working on the scientist boss; create 2 more enemies if done
- Amanda
    - good/evil gameplay tracker system
    - game ending cutscene that takes place after the scientist boss fight + alternate ending
- Tom
    - sound assets and adding sound to places
    - Additional art assets for rooms and enemies
- Sky
    - sound assets and adding sound to places
    - Additional art assets for rooms and enemies

## Milestone 4: Final Game

Week 1

- Vincent
    - BOIDs swarm enemy AI
- Dean
    - new region; regions differ by visuals, enemies, and enemy placements, but are otherwise similar.
- Sky
    - Traps system and assets
    - New region room assets
- Tom
    - Locked rooms that require a key
    - New region room assets
- Amanda

    - NPC rooms
    - New region room assets

Week 2

- Vincent
    - Worm enemy AI for second boss
- Dean
    - Obstacle detection
- Sky
    - Parkour rooms
- Tom
    - Dynamic assets
- Amanda
    - Visual assets and animation for second boss
- Everyone
    - Asset + sound cleanup / finalizing